

第24章 I2C—读写 EEPROM

本章参考资料：《STM32F4xx 参考手册》、《STM32F4xx 规格书》、库帮助文档《stm32f4xx_dsp_stdperiph_lib_um.chm》及《I2C 总线协议》。

若对 I2C 通讯协议不了解，可先阅读《I2C 总线协议》文档的内容学习。若想了解 SMBUS，可阅读《smbus20》文档。

关于 EEPROM 存储器，请参考“常用存储器介绍”章节，实验中的 EEPROM，请参考其规格书《AT24C02》来了解。

24.1 I2C 协议简介

I2C 通讯协议(Inter-Integrated Circuit)是由 Philips 公司开发的，由于它引脚少，硬件实现简单，可扩展性强，不需要 USART、CAN 等通讯协议的外部收发设备，现在被广泛地使用在系统内多个集成电路(IC)间的通讯。

下面我们分别对 I2C 协议的物理层及协议层进行讲解。

24.1.1 I2C 物理层

I2C 通讯设备之间的常用连接方式见图 24-1。

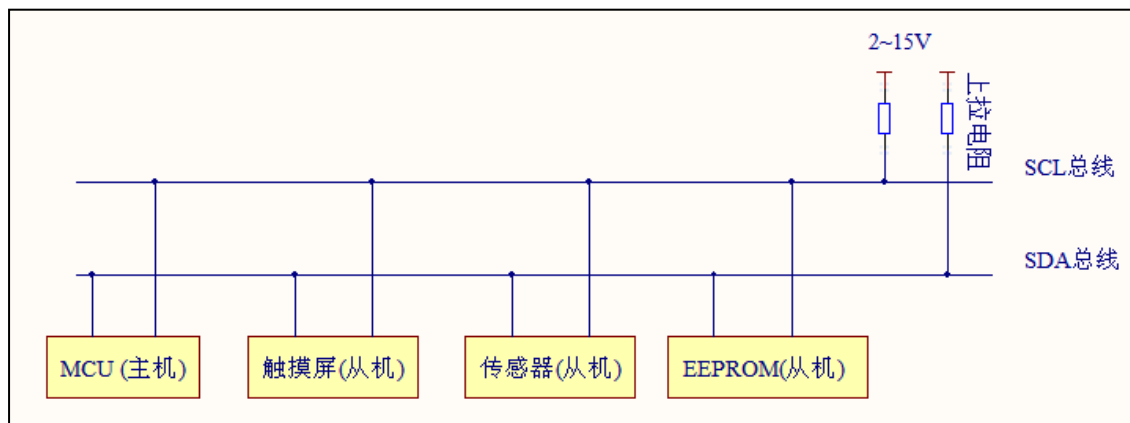


图 24-1 常见的 I2C 通讯系统

它的物理层有如下特点：

- (1) 它是一个支持多设备的总线。“总线”指多个设备共用的信号线。在一个 I2C 通讯总线中，可连接多个 I2C 通讯设备，支持多个通讯主机及多个通讯从机。
- (2) 一个 I2C 总线只使用两条总线线路，一条双向串行数据线(SDA)，一条串行时钟线(SCL)。数据线即用来表示数据，时钟线用于数据收发同步。
- (3) 每个连接到总线的设备都有一个独立的地址，主机可以利用这个地址进行不同设备之间的访问。
- (4) 总线通过上拉电阻接到电源。当 I2C 设备空闲时，会输出高阻态，而当所有设备都空闲，都输出高阻态时，由上拉电阻把总线拉成高电平。

- (5) 多个主机同时使用总线时，为了防止数据冲突，会利用仲裁方式决定由哪个设备占用总线。
- (6) 具有三种传输模式：标准模式传输速率为 100kbit/s，快速模式为 400kbit/s，高速模式下可达 3.4Mbit/s，但目前大多 I²C 设备尚不支持高速模式。
- (7) 连接到相同总线的 IC 数量受到总线的最大电容 400pF 限制。

24.1.2 协议层

I²C 的协议定义了通讯的起始和停止信号、数据有效性、响应、仲裁、时钟同步和地址广播等环节。

1. I²C 基本读写过程

先看看 I²C 通讯过程的基本结构，它的通讯过程见图 24-2、图 24-3 及图 24-4。

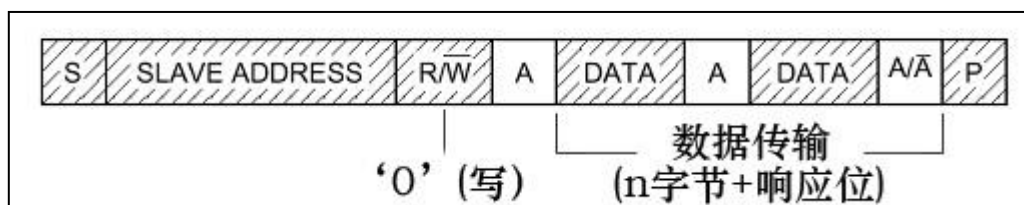


图 24-2 主机写数据到从机

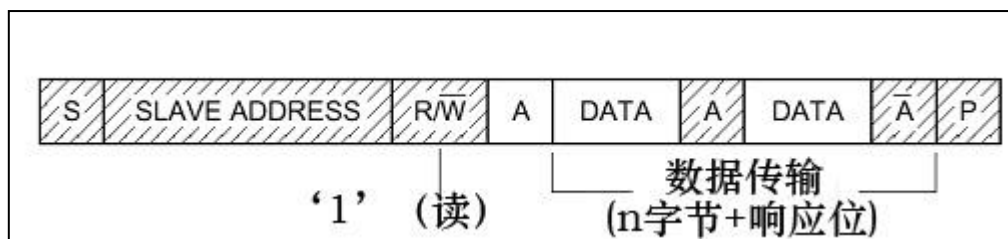


图 24-3 主机由从机中读数据

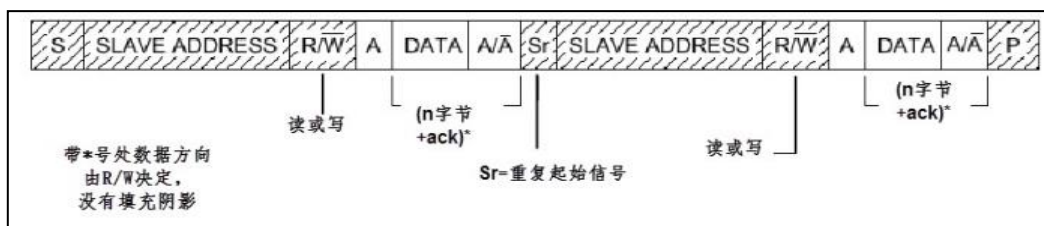


图 24-4 I²C 通讯复合格式

图例：  数据由主机传输至从机 S：传输开始信号

SLAVE_ADDRESS: 从机地址

 数据由从机传输至主机 $\overline{R/W}$ ：传输方向选择位，1 为读，0 为写

$\overline{A/A}$ ：应答(ACK)或非应答(NACK)信号

P：停止传输信号

这些图表示的是主机和从机通讯时，SDA 线的数据包序列。

其中 S 表示由主机的 I2C 接口产生的传输起始信号(S)，这时连接到 I2C 总线上的所有从机都会接收到这个信号。

起始信号产生后，所有从机就开始等待主机紧接下来广播的从机地址信号 (SLAVE_ADDRESS)。在 I2C 总线上，每个设备的地址都是唯一的，当主机广播的地址与某个设备地址相同时，这个设备就被选中了，没被选中的设备将会忽略之后的数据信号。根据 I2C 协议，这个从机地址可以是 7 位或 10 位。

在地址位之后，是传输方向的选择位，该位为 0 时，表示后面的数据传输方向是由主机传输至从机，即主机向从机写数据。该位为 1 时，则相反，即主机由从机读数据。

从机接收到匹配的地址后，主机或从机会返回一个应答(ACK)或非应答(NACK)信号，只有接收到应答信号后，主机才能继续发送或接收数据。

若配置的方向传输位为“写数据”方向，即第一幅图的情况，广播完地址，接收到应答信号后，主机开始正式向从机传输数据(DATA)，数据包的大小为 8 位，主机每发送完一个字节数据，都要等待从机的应答信号(ACK)，重复这个过程，可以向从机传输 N 个数据，这个 N 没有大小限制。当数据传输结束时，主机向从机发送一个停止传输信号(P)，表示不再传输数据。

若配置的方向传输位为“读数据”方向，即第二幅图的情况，广播完地址，接收到应答信号后，从机开始向主机返回数据(DATA)，数据包大小也为 8 位，从机每发送完一个数据，都会等待主机的应答信号(ACK)，重复这个过程，可以返回 N 个数据，这个 N 也没有大小限制。当主机希望停止接收数据时，就向从机返回一个非应答信号(NACK)，则从机自动停止数据传输。

除了基本的读写，I2C 通讯更常用的是复合格式，即第三幅图的情况，该传输过程有两次起始信号(S)。一般在第一次传输中，主机通过 SLAVE_ADDRESS 寻找到从设备后，发送一段“数据”，这段数据通常用于表示从设备内部的寄存器或存储器地址(注意区分它与 SLAVE_ADDRESS 的区别)；在第二次的传输中，对该地址的内容进行读或写。也就是说，第一次通讯是告诉从机读写地址，第二次则是读写的实际内容。

以上通讯流程中包含的各个信号分解如下：

2. 通讯的起始和停止信号

前文中提到的起始(S)和停止(P)信号是两种特殊的状态，见图 24-5。当 SCL 线是高电平时 SDA 线从高电平向低电平切换，这个情况表示通讯的起始。当 SCL 是高电平时 SDA 线由低电平向高电平切换，表示通讯的停止。起始和停止信号一般由主机产生。

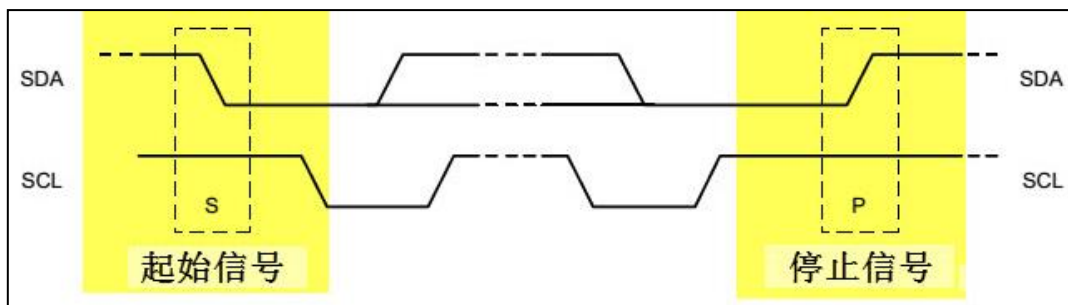


图 24-5 起始和停止信号

3. 数据有效性

I2C 使用 SDA 信号线来传输数据，使用 SCL 信号线进行数据同步。见图 24-6。SDA 数据线在 SCL 的每个时钟周期传输一位数据。传输时，SCL 为高电平的时候 SDA 表示的数据有效，即此时的 SDA 为高电平时表示数据“1”，为低电平时表示数据“0”。当 SCL 为低电平时，SDA 的数据无效，一般在这个时候 SDA 进行电平切换，为下一次表示数据做好准备。

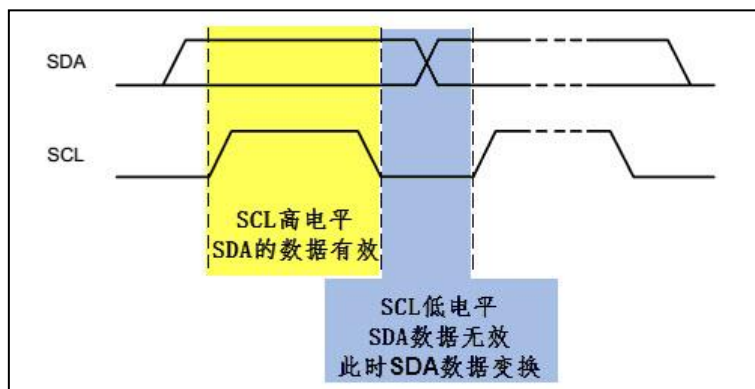


图 24-6 数据有效性

每次数据传输都以字节为单位，每次传输的字节数不受限制。

4. 地址及数据方向

I2C 总线上的每个设备都有自己的独立地址，主机发起通讯时，通过 SDA 信号线发送设备地址(SLAVE_ADDRESS)来查找从机。I2C 协议规定设备地址可以是 7 位或 10 位，实际中 7 位的地址应用比较广泛。紧跟设备地址的一个数据位用来表示数据传输方向，它是数据方向位(R/W)，第 8 位或第 11 位。数据方向位为“1”时表示主机由从机读数据，该位为“0”时表示主机向从机写数据。见图 24-7。

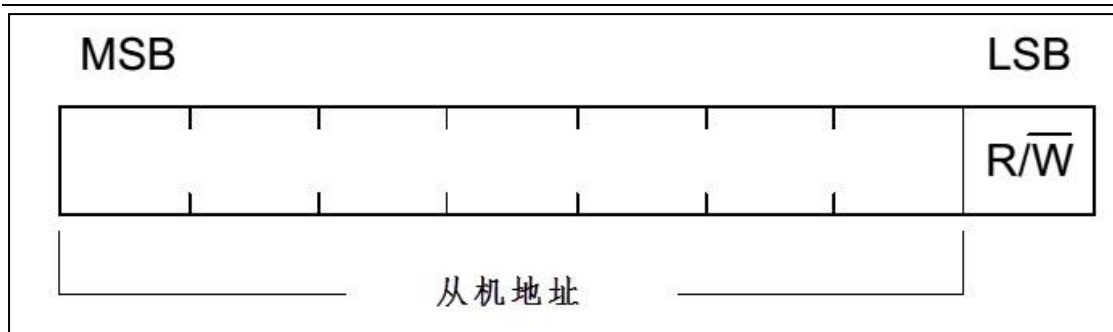


图 24-7 设备地址(7 位)及数据传输方向

读数据方向时，主机会释放对 SDA 信号线的控制，由从机控制 SDA 信号线，主机接收信号，写数据方向时，SDA 由主机控制，从机接收信号。

5. 响应

I2C 的数据和地址传输都带响应。响应包括“应答(ACK)”和“非应答(NACK)”两种信号。作为数据接收端时，当设备(无论主从机)接收到 I2C 传输的一个字节数据或地址后，若希望对方继续发送数据，则需要向对方发送“应答(ACK)”信号，发送方会继续发送下一个数据；若接收端希望结束数据传输，则向对方发送“非应答(NACK)”信号，发送方接收到该信号后会产生一个停止信号，结束信号传输。见图 24-8。

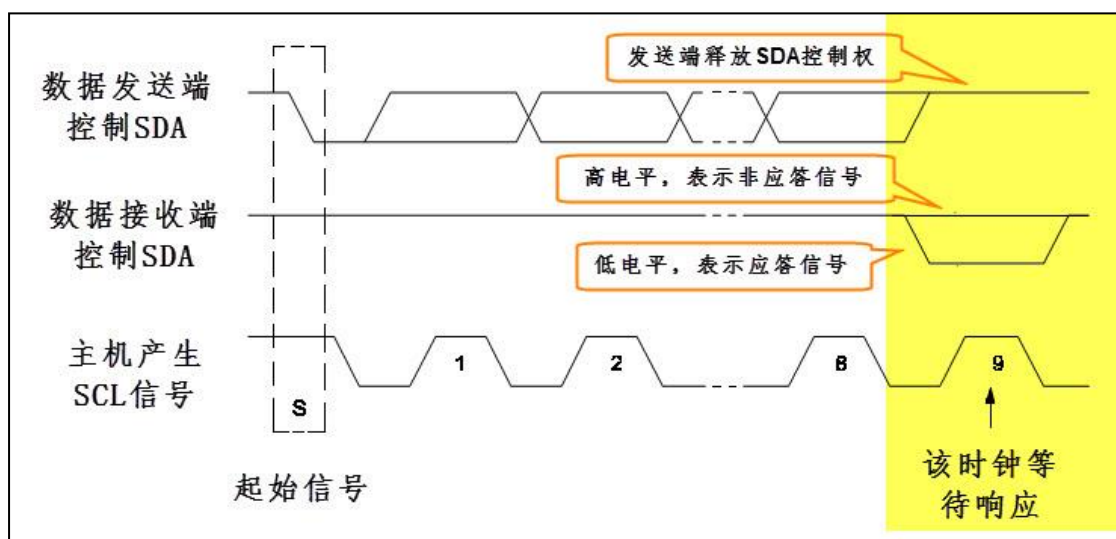


图 24-8 响应与非响应信号

传输时主机产生时钟，在第 9 个时钟时，数据发送端会释放 SDA 的控制权，由数据接收端控制 SDA，若 SDA 为高电平，表示非应答信号(NACK)，低电平表示应答信号(ACK)。

24.2 STM32 的 I2C 特性及架构

如果我们直接控制 STM32 的两个 GPIO 引脚，分别用作 SCL 及 SDA，按照上述信号的时序要求，直接像控制 LED 灯那样控制引脚的输出(若是接收数据时则读取 SDA 电平)，就可以实现 I2C 通讯。同样，假如我们按照 USART 的要求去控制引脚，也能实现 USART

通讯。所以只要遵守协议，就是标准的通讯，不管您如何实现它，不管是 ST 生产的控制器还是 ATMEL 生产的存储器，都能按通讯标准交互。

由于直接控制 GPIO 引脚电平产生通讯时序时，需要由 CPU 控制每个时刻的引脚状态，所以称之为“软件模拟协议”方式。

相对地，还有“硬件协议”方式，STM32 的 I2C 片上外设专门负责实现 I2C 通讯协议，只要配置好该外设，它就会自动根据协议要求产生通讯信号，收发数据并缓存起来，CPU 只要检测该外设的状态和访问数据寄存器，就能完成数据收发。这种由硬件外设处理 I2C 协议的方式减轻了 CPU 的工作，且使软件设计更加简单。

24.2.1 STM32 的 I²C 外设简介

STM32 的 I2C 外设可用作通讯的主机及从机，支持 100Kbit/s 和 400Kbit/s 的速率，支持 7 位、10 位设备地址，支持 DMA 数据传输，并具有数据校验功能。它的 I2C 外设还支持 SMBus2.0 协议，SMBus 协议与 I2C 类似，主要应用于笔记本电脑的电池管理中，本教程不展开，感兴趣的读者可参考《SMBus2.0》文档了解。

24.2.2 STM32 的 I²C 架构剖析

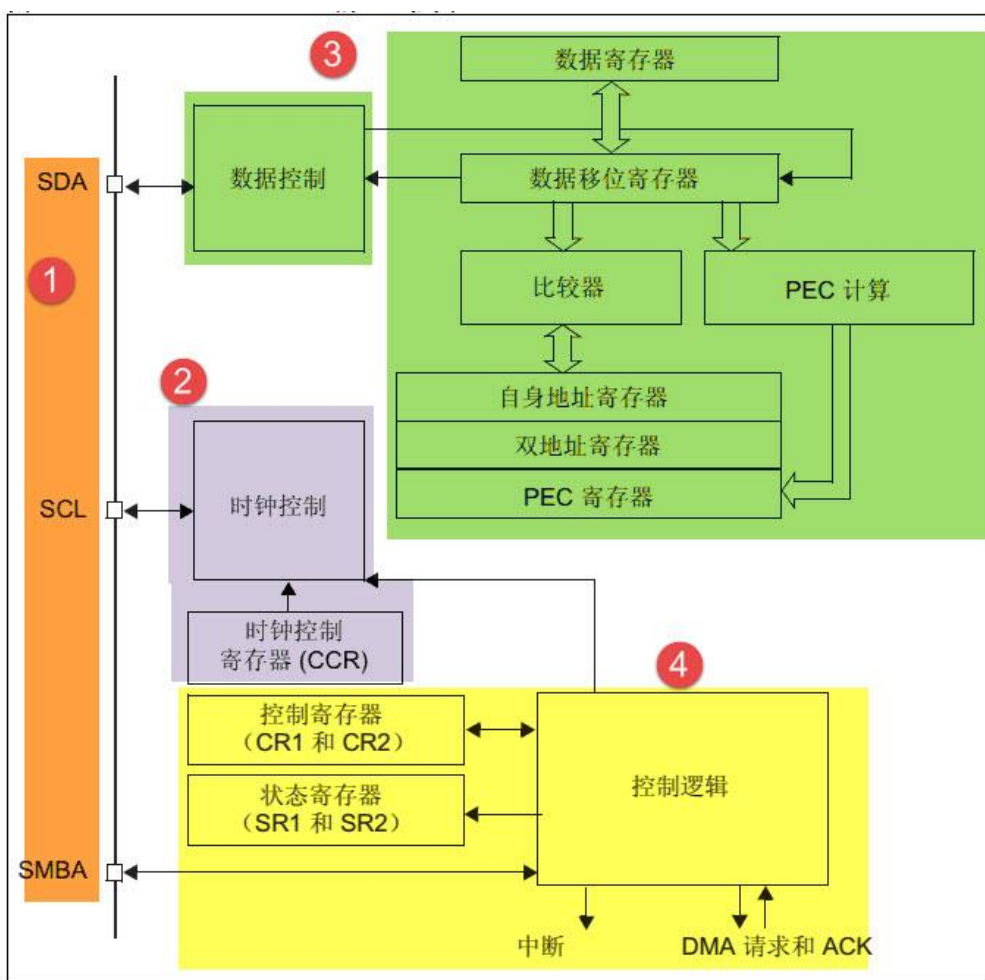


图 24-9 I2C 架构图

1. 通讯引脚

I²C 的所有硬件架构都是根据图中左侧 SCL 线和 SDA 线展开的(其中的 SMBA 线用于 SMBUS 的警告信号, I2C 通讯没有使用)。STM32 芯片有多个 I2C 外设, 它们的 I2C 通讯信号引出到不同的 GPIO 引脚上, 使用时必须配置到这些指定的引脚, 见表 24-1。关于 GPIO 引脚的复用功能, 可查阅《STM32F4xx 规格书》, 以它为准。

表 24-1 STM32F4xx 的 I2C 引脚(整理自《STM32F4xx 规格书》)

引脚	I2C 编号		
	I2C1	I2C2	I2C3
SCL	PB6/PB10	PH4/PF1/PB10	PH7/PA8
SDA	PB7/PB9	PH5/PF0/PB11	PH8/PC9

注: 其中 PF、PH 端口在 176 引脚型号的芯片才有。

2. 时钟控制逻辑

SCL 线的时钟信号, 由 I²C 接口根据时钟控制寄存器(CCR)控制, 控制的参数主要为时钟频率。配置 I2C 的 CCR 寄存器可修改通讯速率相关的参数:

- ❑ 可选择 I2C 通讯的“标准/快速”模式, 这两个模式分别 I2C 对应 100/400Kbit/s 的通讯速率。
- ❑ 在快速模式下可选择 SCL 时钟的占空比, 可选 $T_{low}/T_{high}=2$ 或 $T_{low}/T_{high}=16/9$ 模式, 我们知道 I2C 协议在 SCL 高电平时对 SDA 信号采样, SCL 低电平时 SDA 准备下一个数据, 修改 SCL 的高低电平比会影响数据采样, 但其实这两个模式的比例差别并不大, 若不是要求非常严格, 这里随便选就可以了。
- ❑ CCR 寄存器中还有一个 12 位的配置因子 CCR, 它与 I2C 外设的输入时钟源共同作用, 产生 SCL 时钟, STM32 的 I2C 外设都挂载在 APB1 总线上, 使用 APB1 的时钟源 PCLK1, SCL 信号线的输出时钟公式如下:

标准模式:

$$T_{high}=CCR*T_{PCLK1}$$

$$T_{low}=CCR*T_{PCLK1}$$

快速模式中 $T_{low}/T_{high}=2$ 时:

$$T_{high}=CCR*T_{PCLK1}$$

$$T_{low}=2*CCR*T_{PCLK1}$$

快速模式中 $T_{low}/T_{high}=16/9$ 时:

$$T_{high}=9*CCR*T_{PCLK1}$$

$$T_{low}=16*CCR*T_{PCLK1}$$

例如，我们的 PCLK1=42MHz，想要配置 400Kbit/s 的速率，计算方式如下：

PCLK 时钟周期：TPCLK1 = 1/42000000

目标 SCL 时钟周期：TSCL = 1/400000

SCL 时钟周期内的高电平时间：THIGH = TSCL/3

SCL 时钟周期内的低电平时间：TLOW = 2*TSCL/3

计算 CCR 的值：CCR = THIGH/TPCLK1 = 35

该结果刚好为整数，所以我们可直接把 CCR 取值为 35，这样 I2C 的 SCL 实际频率即为 400KHz。特别地，CCR 寄存器是无法配置小数参数的，如果配置某个速率算出来 CCR 的结果为小数的话，需要对结果进行取整，再配置，取整后 SCL 的输出频率会跟原目标频率稍微不同，取整后除了通讯稍慢或稍快一点以外，不会对 I2C 的标准通讯造成其它影响。

3. 数据控制逻辑

I2C 的 SDA 信号主要连接到数据移位寄存器上，数据移位寄存器的数据来源及目标是数据寄存器(DR)、地址寄存器(OAR)、PEC 寄存器以及 SDA 数据线。当向外发送数据的时候，数据移位寄存器以“数据寄存器”为数据源，把数据一位一位地通过 SDA 信号线发送出去；当从外部接收数据的时候，数据移位寄存器把 SDA 信号线采样到的数据一位一位地存储到“数据寄存器”中。若使能了数据校验，接收到的数据会经过 PCE 计算器运算，运算结果存储在“PEC 寄存器”中。当 STM32 的 I2C 工作在从机模式的时候，接收到设备地址信号时，数据移位寄存器会把接收到的地址与 STM32 的自身的“I2C 地址寄存器”的值作比较，以便响应主机的寻址。STM32 的自身 I2C 地址可通过修改“自身地址寄存器”修改，支持同时使用两个 I2C 设备地址，两个地址分别存储在 OAR1 和 OAR2 中。

4. 整体控制逻辑

整体控制逻辑负责协调整个 I2C 外设，控制逻辑的工作模式根据我们配置的“控制寄存器(CR1/CR2)”的参数而改变。在外设工作时，控制逻辑会根据外设的工作状态修改“状态寄存器(SR1 和 SR2)”，我们只要读取这些寄存器相关的寄存器位，就可以了解 I2C 的工作状态了。除此之外，控制逻辑还根据要求，负责控制产生 I2C 中断信号、DMA 请求及各种 I2C 的通讯信号(起始、停止、响应信号等)。

24.2.3 通讯过程

使用 I2C 外设通讯时，在通讯的不同阶段它会对“状态寄存器(SR1 及 SR2)”的不同数据位写入参数，我们通过读取这些寄存器标志来了解通讯状态。

1. 主发送器

见图 24-10。图中的是“主发送器”流程，即作为 I2C 通讯的主机端时，向外发送数据时的过程。

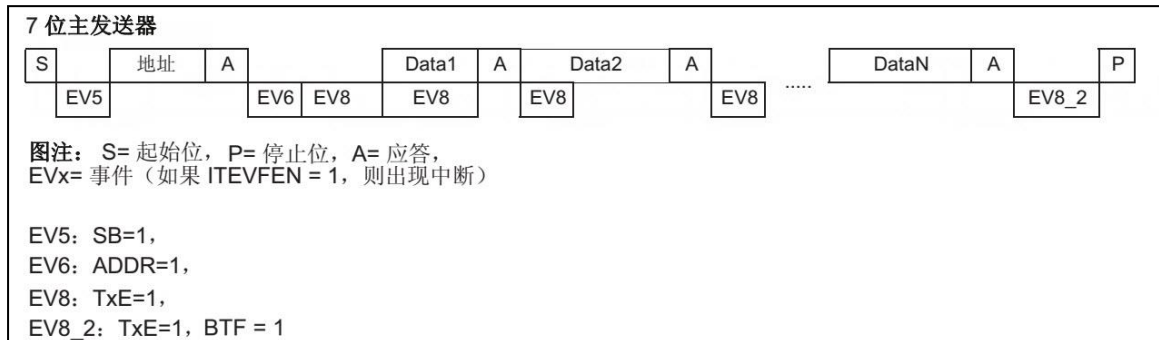


图 24-10 主发送器通讯过程

主发送器发送流程及事件说明如下：

- (1) 控制产生起始信号(S)，当发生起始信号后，它产生事件“EV5”，并会对 SR1 寄存器的“SB”位置 1，表示起始信号已经发送；
- (2) 紧接着发送设备地址并等待应答信号，若有从机应答，则产生事件“EV6”及“EV8”，这时 SR1 寄存器的“ADDR”位及“TXE”位被置 1，ADDR 为 1 表示地址已经发送，TXE 为 1 表示数据寄存器为空；
- (3) 以上步骤正常执行并对 ADDR 位清零后，我们往 I2C 的“数据寄存器 DR”写入要发送的数据，这时 TXE 位会被重置 0，表示数据寄存器非空，I2C 外设通过 SDA 信号线一位位把数据发送出去后，又会产生“EV8”事件，即 TXE 位被置 1，重复这个过程，就可以发送多个字节数据了；
- (4) 当我们发送数据完成后，控制 I2C 设备产生一个停止信号(P)，这个时候会产生 EV2 事件，SR1 的 TXE 位及 BTF 位都被置 1，表示通讯结束。

假如我们使能了 I2C 中断，以上所有事件产生时，都会产生 I2C 中断信号，进入同一个中断服务函数，到 I2C 中断服务程序后，再通过检查寄存器位来了解是哪一个事件。

2. 主接收器

再来分析主接收器过程，即作为 I2C 通讯的主机端时，从外部接收数据的过程，见图 24-11。

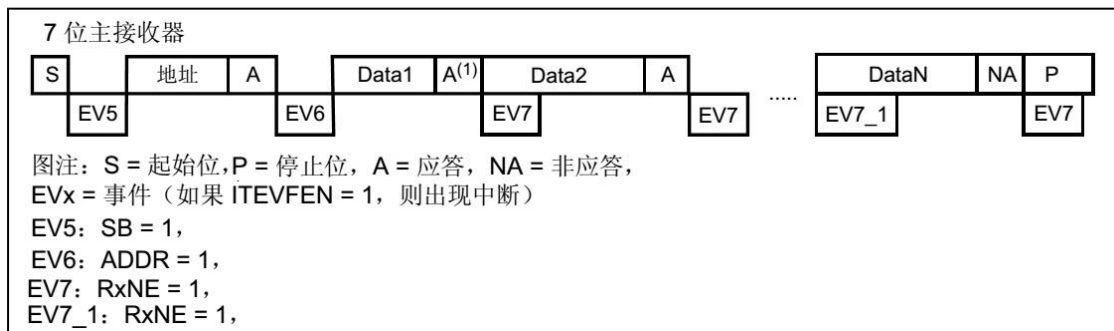


图 24-11 主接收器过程

主接收器接收流程及事件说明如下：

- (1) 同主发送流程，起始信号(S)是由主机端产生的，控制发生起始信号后，它产生事件“EV5”，并会对 SR1 寄存器的“SB”位置 1，表示起始信号已经发送；
- (2) 紧接着发送设备地址并等待应答信号，若有从机应答，则产生事件“EV6”这时 SR1 寄存器的“ADDR”位被置 1，表示地址已经发送。
- (3) 从机端接收到地址后，开始向主机端发送数据。当主机接收到这些数据后，会产生“EV7”事件，SR1 寄存器的 RXNE 被置 1，表示接收数据寄存器非空，我们读取该寄存器后，可对数据寄存器清空，以便接收下一次数据。此时我们可以控制 I2C 发送应答信号(ACK)或非应答信号(NACK)，若应答，则重复以上步骤接收数据，若非应答，则停止传输；
- (4) 发送非应答信号后，产生停止信号(P)，结束传输。

在发送和接收过程中，有的事件不只是标志了我们上面提到的状态位，还可能同时标志主机状态之类的状态位，而且读了之后还需要清除标志位，比较复杂。我们可使用 STM32 标准库函数来直接检测这些事件的复合标志，降低编程难度。

24.3 I2C 初始化结构体详解

跟其它外设一样，STM32 标准库提供了 I2C 初始化结构体及初始化函数来配置 I2C 外设。初始化结构体及函数定义在库文件“stm32f4xx_i2c.h”及“stm32f4xx_i2c.c”中，编程时我们可以结合这两个文件内的注释使用或参考库帮助文档。了解初始化结构体后我们就能对 I2C 外设运用自如了，见代码清单 24-1。

代码清单 24-1 I2C 初始化结构体

```
1 typedef struct {  
2     uint32_t I2C_ClockSpeed; /*!< 设置 SCL 时钟频率，此值要低于 40 0000*/  
3     uint16_t I2C_Mode; /*!< 指定工作模式，可选 I2C 模式及 SMBUS 模式 */  
4     uint16_t I2C_DutyCycle; /*指定时钟占空比，可选 low/high = 2:1 及 16:9 模式*/  
5     uint16_t I2C_OwnAddress1; /*!< 指定自身的 I2C 设备地址 */  
6     uint16_t I2C_Ack; /*!< 使能或关闭响应(一般都要使能) */  
7     uint16_t I2C_AcknowledgedAddress; /*!< 指定地址的长度，可为 7 位及 10 位 */  
8 } I2C_InitTypeDef;
```

这些结构体成员说明如下，其中括号内的文字是对应参数在 STM32 标准库中定义的宏：

(1) I2C_ClockSpeed

本成员设置的是 I2C 的传输速率，在调用初始化函数时，函数会根据我们输入的数值经过运算后把时钟因子写入到 I2C 的时钟控制寄存器 CCR。而我们写入的这个参数值不得高于 400KHz。由于 CCR 寄存器不能写入小数类型的时钟因子，固件库计算 CCR 值时会向下取整，影响到 SCL 的实际频率可能会低于本成员设置的参数值，这时除了通讯稍慢一点以外，不会对 I2C 的标准通讯造成其它影响。

(2) I2C_Mode

本成员是选择 I²C 的使用方式，有 I²C 模式(I2C_Mode_I2C)和 SMBus 主、从模式(I2C_Mode_SMBusHost、 I2C_Mode_SMBusDevice)。I2C 不需要在此处区分主从模式，直接设置 I2C_Mode_I2C 即可。

(3) I2C_DutyCycle

本成员设置的是 I²C 的 SCL 线时钟的占空比。该配置有两个选择，分别为低电平时间比高电平时间为 2: 1 (I2C_DutyCycle_2)和 16: 9 (I2C_DutyCycle_16_9)。其实这两个模式的比例差别并不大，一般要求都不会如此严格，这里随便选就可以了。

(4) I2C_OwnAddress1

本成员配置的是 STM32 的 I2C 设备自己的地址，每个连接到 I2C 总线上的设备都要有一个自己的地址，作为主机也不例外。地址可设置为 7 位或 10 位(受下面 I2C_AcknowledgeAddress 成员决定)，只要该地址是 I2C 总线上唯一的即可。

STM32 的 I2C 外设可同时使用两个地址，即同时对两个地址作出响应，这个结构成员 I2C_OwnAddress1 配置的是默认的、OAR1 寄存器存储的地址，若需要设置第二个地址寄存器 OAR2，可使用 I2C_OwnAddress2Config 函数来配置，OAR2 不支持 10 位地址。

(5) I2C_Ack_Enable

本成员是关于 I²C 应答设置，设置为使能则可以发送响应信号。该成员值一般配置为允许应答(I2C_Ack_Enable)，这是绝大多数遵循 I²C 标准的设备的通讯要求，改为禁止应答(I2C_Ack_Disable)往往会导致通讯错误。

(6) I2C_AcknowledgeAddress

本成员选择 I2C 的寻址模式是 7 位还是 10 位地址。这需要根据实际连接到 I2C 总线上设备的地址进行选择，这个成员的配置也影响到 I2C_OwnAddress1 成员，只有这里设置成 10 位模式时，I2C_OwnAddress1 才支持 10 位地址。

配置完这些结构体成员值，调用库函数 I2C_Init 即可把结构体的配置写入到寄存器中。

24.4 I2C—读写 EEPROM 实验

EEPROM 是一种掉电后数据不丢失的存储器，常用来存储一些配置信息，以便系统重新上电的时候加载之。EEPOM 芯片最常用的通讯方式就是 I²C 协议，本小节以 EEPROM 的读写实验为大家讲解 STM32 的 I²C 使用方法。实验中 STM32 的 I2C 外设采用主模式，分别用作主发送器和主接收器，通过查询事件的方式来确保正常通讯。

24.4.1 硬件设计

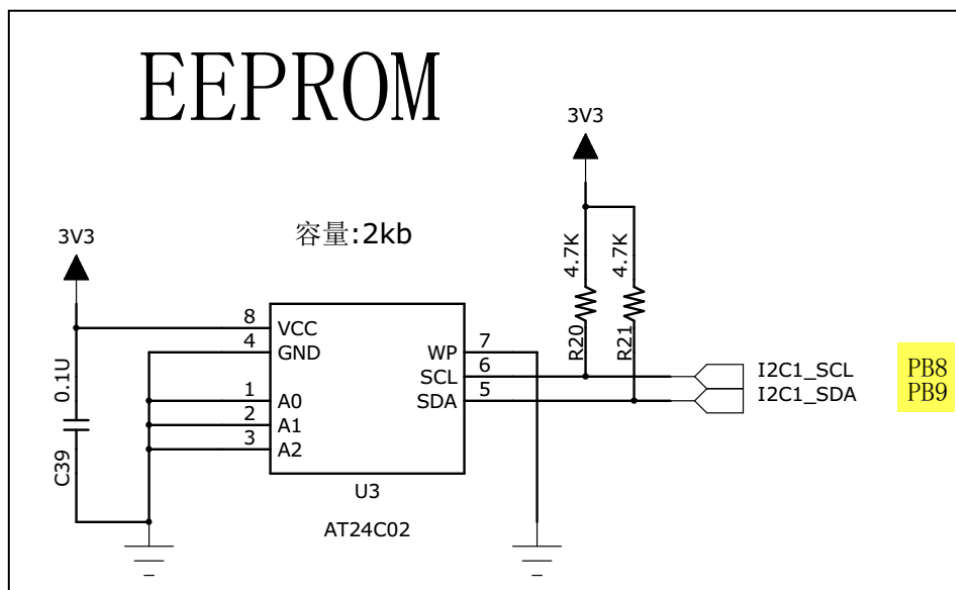


图 24-12 EEPROM 硬件连接图

本实验板中的 EEPROM 芯片(型号：AT24C02)的 SCL 及 SDA 引脚连接到了 STM32 对应的 I2C 引脚中，结合上拉电阻，构成了 I2C 通讯总线，它们通过 I2C 总线交互。EEPROM 芯片的设备地址一共有 7 位，其中高 4 位固定为：1010 b，低 3 位则由 A0/A1/A2 信号线的电平决定，见图 24-13，图中的 R/W 是读写方向位，与地址无关。

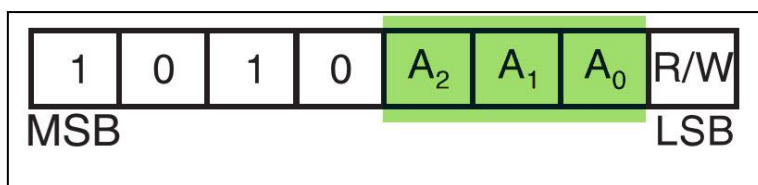


图 24-13 EEPROM 设备地址(摘自《AT24C02》规格书)

按照我们此处的连接，A0/A1/A2 均为 0，所以 EEPROM 的 7 位设备地址是：1010000b，即 0x50。由于 I2C 通讯时常常是地址跟读写方向连在一起构成一个 8 位数，且当 R/W 位为 0 时，表示写方向，所以加上 7 位地址，其值为“0xA0”，常称该值为 I2C 设备的“写地址”；当 R/W 位为 1 时，表示读方向，加上 7 位地址，其值为“0xA1”，常称该值为“读地址”。

EEPROM 芯片中还有一个 WP 引脚，具有写保护功能，当该引脚电平为高时，禁止写入数据，当引脚为低电平时，可写入数据，此处直接接地，不使用写保护功能。

关于 EEPROM 的更多信息，可参考其数据手册《AT24C02》来了解。若您使用的实验板 EEPROM 的型号、设备地址或控制引脚不一样，只需根据我们的工程修改即可，程序的控制原理相同。

24.4.2 软件设计

为了使工程更加有条理，我们把读写 EEPROM 相关的代码独立分开存储，方便以后移植。在“工程模板”之上新建“bsp_i2c_ee.c”及“bsp_i2c_ee.h”文件，这些文件也可根据您的喜好命名，它们不属于 STM32 标准库的内容，是由我们自己根据应用需要编写的。

1. 编程要点

- (1) 配置通讯使用的目标引脚为开漏模式；
- (2) 使能 I2C 外设的时钟；
- (3) 配置 I2C 外设的模式、地址、速率等参数并使能 I2C 外设；
- (4) 编写基本 I2C 按字节收发的函数；
- (5) 编写读写 EEPROM 存储内容的函数；
- (6) 编写测试程序，对读写数据进行校验。

2. 代码分析

I2C 硬件相关宏定义

我们把 I2C 硬件相关的配置都以宏的形式定义到“bsp_i2c_ee.h”文件中，见代码清单 24-2。

代码清单 24-2 I2C 硬件配置相关的宏

```
2 /* STM32 I2C 速率 */
3 #define I2C_Speed          400000
4
5 /* STM32 自身的 I2C 地址，这个地址只要与 STM32 外挂的 I2C 器件地址不一样即可 */
6 #define I2C_OWN_ADDRESS7   0X0A
7
8 /*I2C 接口*/
9 #define EEPROM_I2C          I2C1
10 #define EEPROM_I2C_CLK      RCC_APB1Periph_I2C1
11
12 #define EEPROM_I2C_SCL_PIN   GPIO_Pin_8
13 #define EEPROM_I2C_SCL_GPIO_PORT  GPIOB
14 #define EEPROM_I2C_SCL_GPIO_CLK  RCC_AHB1Periph_GPIOB
15 #define EEPROM_I2C_SCL_SOURCE  GPIO_PinSource8
16 #define EEPROM_I2C_SCL_AF    GPIO_AF_I2C1
17
18 #define EEPROM_I2C_SDA_PIN   GPIO_Pin_9
19 #define EEPROM_I2C_SDA_GPIO_PORT  GPIOB
20 #define EEPROM_I2C_SDA_GPIO_CLK  RCC_AHB1Periph_GPIOB
21 #define EEPROM_I2C_SDA_SOURCE  GPIO_PinSource9
22 #define EEPROM_I2C_SDA_AF    GPIO_AF_I2C1
```

以上代码根据硬件连接，把与 EEPROM 通讯使用的 I2C 号、引脚号、引脚源以及复用功能映射都以宏封装起来，并且定义了自身的 I2C 地址及通讯速率，以便配置模式的时候使用。

初始化 I2C 的 GPIO

利用上面的宏，编写 I2C GPIO 引脚的初始化函数，见代码清单 13-2。

代码清单 24-3 I2C 初始化函数

```
2 /**
```

```
3  * @brief I2C1 I/O 配置
4  * @param 无
5  * @retval 无
6  */
7  static void I2C_GPIO_Config(void)
8  {
9      GPIO_InitTypeDef GPIO_InitStructure;
10
11     /*使能 I2C 外设时钟 */
12     RCC_APB1PeriphClockCmd(EEPROM_I2C_CLK, ENABLE);
13
14     /*使能 I2C 引脚的 GPIO 时钟*/
15     RCC_AHB1PeriphClockCmd(EEPROM_I2C_SCL_GPIO_CLK |
16                             EEPROM_I2C_SDA_GPIO_CLK, ENABLE);
17
18     /* 连接引脚源 Pxx 到 I2C_SCL*/
19     GPIO_PinAFConfig(EEPROM_I2C_SCL_GPIO_PORT, EEPROM_I2C_SCL_SOURCE,
20                      EEPROM_I2C_SCL_AF);
21     /* 连接引脚源 Pxx 到 I2C_SDA*/
22     GPIO_PinAFConfig(EEPROM_I2C_SDA_GPIO_PORT, EEPROM_I2C_SDA_SOURCE,
23                      EEPROM_I2C_SDA_AF);
24
25     /*配置 SCL 引脚 */
26     GPIO_InitStructure.GPIO_Pin = EEPROM_I2C_SCL_PIN;
27     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;
28     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
29     GPIO_InitStructure.GPIO_OType = GPIO_OType_OD;
30     GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;
31     GPIO_Init(EEPROM_I2C_SCL_GPIO_PORT, &GPIO_InitStructure);
32
33     /*配置 SDA 引脚 */
34     GPIO_InitStructure.GPIO_Pin = EEPROM_I2C_SDA_PIN;
35     GPIO_Init(EEPROM_I2C_SDA_GPIO_PORT, &GPIO_InitStructure);
36 }
```

同为外设使用的 GPIO 引脚初始化，初始化的流程与“串口初始化函数”章节中的类似，主要区别是引脚的模式。函数执行流程如下：

- (1) 使用 GPIO_InitTypeDef 定义 GPIO 初始化结构体变量，以便下面用于存储 GPIO 配置；
- (2) 调用库函数 RCC_APB1PeriphClockCmd 使能 I2C 外设时钟，调用 RCC_AHB1PeriphClockCmd 来使能 I2C 引脚使用的 GPIO 端口时钟，调用时使用“|”操作同时配置两个引脚。
- (3) 向 GPIO 初始化结构体赋值，把引脚初始化成复用开漏模式，要注意 I2C 的引脚必须使用这种模式。
- (4) 使用以上初始化结构体的配置，调用 GPIO_Init 函数向寄存器写入参数，完成 GPIO 的初始化。

配置 I2C 的模式

以上只是配置了 I2C 使用的引脚，还不算对 I2C 模式的配置，见代码清单 24-4。

代码清单 24-4 配置 I2C 模式

```
2 /**
3  * @brief I2C 工作模式配置
4  * @param 无
5  * @retval 无
6  */
7  static void I2C_Mode_Config(void)
8  {
```

```
9      I2C_InitTypeDef  I2C_InitStructure;
10
11      /* I2C 配置 */
12      /*I2C 模式*/
13      I2C_InitStructure.I2C_Mode = I2C_Mode_I2C;
14      /*占空比*/
15      I2C_InitStructure.I2C_DutyCycle = I2C_DutyCycle_2;
16      /*I2C 自身地址*/
17      I2C_InitStructure.I2C_OwnAddress1 =I2C_OWN_ADDRESS7;
18      /*使能响应*/
19      I2C_InitStructure.I2C_Ack = I2C_Ack_Enable ;
20      /* I2C 的寻址模式 */
21      I2C_InitStructure.I2C_AcknowledgedAddress = I2C_AcknowledgedAddress_7bit;
22      /* 通信速率 */
23      I2C_InitStructure.I2C_ClockSpeed = I2C_Speed;
24      /*写入配置*/
25      I2C_Init(EEPROM_I2C, &I2C_InitStructure);
26      /* 使能 I2C */
27      I2C_Cmd(EEPROM_I2C, ENABLE);
28 }
29
30 /**
31  * @brief  I2C 外设初始化
32  * @param  无
33  * @retval 无
34  */
35 void I2C_EE_Init(void)
36 {
37     I2C_GPIO_Config();
38
39     I2C_Mode_Config();
40 }
```

熟悉 STM32 I2C 结构的话，这段初始化程序就十分好理解了，它把 I2C 外设通讯时钟 SCL 的低/高电平比设置为 2，使能响应功能，使用 7 位地址 I2C_OWN_ADDRESS7 以及速率配置为 I2C_Speed(前面在 bsp_i2c_ee.h 定义的宏)。最后调用库函数 I2C_Init 把这些配置写入寄存器，并调用 I2C_Cmd 函数使能外设。

为方便调用，我们把 I2C 的 GPIO 及模式配置都用 I2C_EE_Init 函数封装起来。

向 EEPROM 写入一个字节的的数据

初始化好 I2C 外设后，就可以使用 I2C 通讯了，我们看看如何向 EEPROM 写入一个字节的的数据，见代码清单 24-5。

代码清单 24-5 向 EEPROM 写入一个字节的的数据

```
1
2  /*****
3  /*通讯等待超时时间*/
4  #define I2CT_FLAG_TIMEOUT      ((uint32_t)0x1000)
5  #define I2CT_LONG_TIMEOUT      ((uint32_t)(10 * I2CT_FLAG_TIMEOUT))
6
7  /**
8   * @brief  I2C 等待事件超时的情况下会调用这个函数来处理
9   * @param  errorCode: 错误代码，可以用来定位是哪个环节出错。
10  * @retval 返回 0，表示 IIC 读取失败。
11  */
12 static uint32_t I2C_TIMEOUT_UserCallback(uint8_t errorCode)
13 {
14     /* 使用串口 printf 输出错误信息，方便调试 */
15     EEPROM_ERROR("I2C 等待超时!errorCode = %d",errorCode);
16     return 0;
17 }
```

```
18 /**
19  * @brief   写一个字节到 I2C EEPROM 中
20  * @param   pBuffer:缓冲区指针
21  * @param   WriteAddr:写地址
22  * @retval   正常返回 1, 异常返回 0
23  */
24 uint32_t I2C_EE_ByteWrite(u8* pBuffer, u8 WriteAddr)
25 {
26     /* 产生 I2C 起始信号 */
27     I2C_GenerateSTART(EEPROM_I2C, ENABLE);
28
29     /*设置超时等待时间*/
30     I2CTimeout = I2CT_FLAG_TIMEOUT;
31     /* 检测 EV5 事件并清除标志*/
32     while (!I2C_CheckEvent(EEPROM_I2C, I2C_EVENT_MASTER_MODE_SELECT))
33     {
34         if ((I2CTimeout--) == 0) return I2C_TIMEOUT_UserCallback(0);
35     }
36
37     /* 发送 EEPROM 设备地址 */
38     I2C_Send7bitAddress(EEPROM_I2C, EEPROM_ADDRESS,
39                         I2C_Direction_Transmitter);
40
41     I2CTimeout = I2CT_FLAG_TIMEOUT;
42     /* 检测 EV6 事件并清除标志*/
43     while (!I2C_CheckEvent(EEPROM_I2C,
44                             I2C_EVENT_MASTER_TRANSMITTER_MODE_SELECTED))
45     {
46         if ((I2CTimeout--) == 0) return I2C_TIMEOUT_UserCallback(1);
47     }
48
49     /* 发送要写入的 EEPROM 内部地址(即 EEPROM 内部存储器的地址) */
50     I2C_SendData(EEPROM_I2C, WriteAddr);
51
52     I2CTimeout = I2CT_FLAG_TIMEOUT;
53     /* 检测 EV8 事件并清除标志*/
54     while (!I2C_CheckEvent(EEPROM_I2C,
55                             I2C_EVENT_MASTER_BYTE_TRANSMITTED))
56     {
57         if ((I2CTimeout--) == 0) return I2C_TIMEOUT_UserCallback(2);
58     }
59     /* 发送一字节要写入的数据 */
60     I2C_SendData(EEPROM_I2C, *pBuffer);
61
62     I2CTimeout = I2CT_FLAG_TIMEOUT;
63     /* 检测 EV8 事件并清除标志*/
64     while (!I2C_CheckEvent(EEPROM_I2C,
65                             I2C_EVENT_MASTER_BYTE_TRANSMITTED))
66     {
67         if ((I2CTimeout--) == 0) return I2C_TIMEOUT_UserCallback(3);
68     }
69
70     /* 发送停止信号 */
71     I2C_GenerateSTOP(EEPROM_I2C, ENABLE);
72
73     return 1;
74 }
```

先来分析 I2C_TIMEOUT_UserCallback 函数，它的函数体里只调用了宏 EEPROM_ERROR，这个宏封装了 printf 函数，方便使用串口向上位机打印调试信息，阅读代码时把它当成 printf 函数即可。在 I2C 通讯的很多过程，都需要检测事件，当检测到某事件后才能继续下一步的操作，但有时通讯错误或者 I2C 总线被占用，我们不能无休止地等待下去，所以我们设定每个事件检测都有等待的时间上限，若超过这个时间，我们就

调用 I2C_TIMEOUT_UserCallback 函数输出调试信息(或可以自己加其它操作), 并终止 I2C 通讯。

了解了这个机制, 再来分析 I2C_EE_ByteWrite 函数, 这个函数实现了前面讲的 I2C [主发送器通讯流程](#):

- (1) 使用库函数 I2C_GenerateSTART 产生 I2C 起始信号, 其中的 EEPROM_I2C 宏是前面硬件定义相关的 I2C 编号;
- (2) 对 I2CTimeout 变量赋值为宏 I2CT_FLAG_TIMEOUT, 这个 I2CTimeout 变量在下面的 while 循环中每次循环减 1, 该循环通过调用库函数 I2C_CheckEvent 检测事件, 若检测到事件, 则进入通讯的下一阶段, 若未检测到事件则停留在此处一直检测, 当检测 I2CT_FLAG_TIMEOUT 次都还没等待到事件则认为通讯失败, 调用前面的 I2C_TIMEOUT_UserCallback 输出调试信息, 并退出通讯;
- (3) 调用库函数 I2C_Send7bitAddress 发送 EEPROM 的设备地址, 并把数据传输方向设置为 I2C_Direction_Transmitter(即发送方向), 这个数据传输方向就是通过设置 I2C 通讯中紧跟地址后面的 R/W 位实现的。发送地址后以同样的方式检测 EV6 标志;
- (4) 调用库函数 I2C_SendData 向 EEPROM 发送要写入的内部地址, 该地址是 I2C_EE_ByteWrite 函数的输入参数, 发送完毕后等待 EV8 事件。要注意这个内部地址跟上面的 EEPROM 地址不一样, 上面的是指 I2C 总线设备的独立地址, 而此处的内部地址是指 EEPROM 内数据组织的地址, 也可理解为 EEPROM 内存的地址或 I2C 设备的寄存器地址;
- (5) 调用库函数 I2C_SendData 向 EEPROM 发送要写入的数据, 该数据是 I2C_EE_ByteWrite 函数的输入参数, 发送完毕后等待 EV8 事件;
- (6) 一个 I2C 通讯过程完毕, 调用 I2C_GenerateSTOP 发送停止信号。

在这个通讯过程中, STM32 实际上通过 I2C 向 EEPROM 发送了两个数据, 但为何第一个数据被解释为 EEPROM 的内存地址? 这是由 EEPROM 的自己定义的单字节写入时序, 见图 24-14。

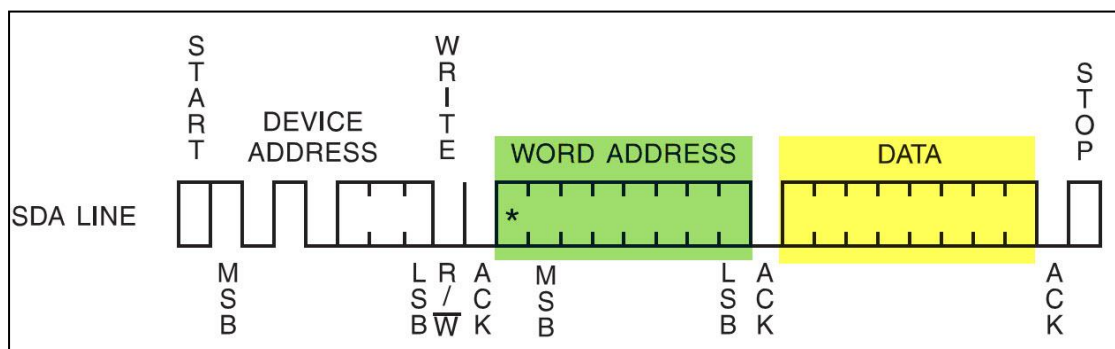


图 24-14 EEPROM 单字节写入时序(摘自《AT24C02》规格书)

EEPROM 的单字节时序规定, 向它写入数据的时候, 第一个字节为内存地址, 第二个字节是要写入的数据内容。所以我们需要理解: 命令、地址的本质都是数据, 对数据的解释不同, 它就有了不同的功能。

多字节写入及状态等待

单字节写入通讯结束后，EEPROM 芯片会根据这个通讯结果擦写该内存地址的内容，这需要一段时间，所以我们在多次写入数据时，要先等待 EEPROM 内部擦写完毕。多个数据写入过程见代码清单 24-6。

代码清单 24-6 多字节写入

```
1 /**
2  * @brief  将缓冲区中的数据写到 I2C EEPROM 中，采用单字节写入的方式，
3           速度比页写入慢
4  * @param pBuffer:缓冲区指针
5  * @param WriteAddr:写地址
6  * @param NumByteToWrite:写的字节数
7  * @retval 无
8  */
9 uint8_t I2C_EE_ByetsWrite(uint8_t* pBuffer,uint8_t WriteAddr,
10                          uint16_t NumByteToWrite)
11 {
12     uint16_t i;
13     uint8_t res;
14
15     /*每写一个字节调用一次 I2C_EE_ByteWrite 函数*/
16     for (i=0; i<NumByteToWrite; i++)
17     {
18         /*等待 EEPROM 准备完毕*/
19         I2C_EE_WaitEepromStandbyState();
20         /*按字节写入数据*/
21         res = I2C_EE_ByteWrite(pBuffer++,WriteAddr++);
22     }
23     return res;
24 }
```

这段代码比较简单，直接使用 for 循环调用前面定义的 I2C_EE_ByteWrite 函数一个字节一个字节地向 EEPROM 发送要写入的数据。在每次数据写入通讯前调用了 I2C_EE_WaitEepromStandbyState 函数等待 EEPROM 内部擦写完毕，该函数的定义见代码清单 24-7。

代码清单 24-7 等待 EEPROM 处于准备状态

```
1 //等待 Standby 状态的最大次数
2 #define MAX_TRIAL_NUMBER 300
3 /**
4  * @brief  等待 EEPROM 到准备状态
5  * @param  无
6  * @retval 正常返回 1，异常返回 0
7  */
8 static uint8_t I2C_EE_WaitEepromStandbyState(void)
9 {
10     __IO uint16_t tmpSR1 = 0;
11     __IO uint32_t EETrials = 0;
12
13     /*总线忙时等待 */
14     I2CTimeout = I2CT_LONG_TIMEOUT;
15     while (I2C_GetFlagStatus(EEPROM_I2C, I2C_FLAG_BUSY))
16     {
17         if ((I2CTimeout--)== 0) return I2C_TIMEOUT_UserCallback(20);
18     }
```

```
19
20     /* 等待从机应答, 最多等待 300 次 */
21     while (1)
22     {
23         /*开始信号 */
24         I2C_GenerateSTART(EEPROM_I2C, ENABLE);
25
26         /* 检测 EV5 事件并清除标志*/
27         I2CTimeout = I2CT_FLAG_TIMEOUT;
28         while (!I2C_CheckEvent(EEPROM_I2C, I2C_EVENT_MASTER_MODE_SELECT))
29         {
30             if ((I2CTimeout--) == 0) return I2C_TIMEOUT_UserCallback(21);
31         }
32
33         /* 发送 EEPROM 设备地址 */
34         I2C_Send7bitAddress(EEPROM_I2C, EEPROM_ADDRESS, I2C_Direction_Transmitter);
35
36         /* 等待 ADDR 标志 */
37         I2CTimeout = I2CT_LONG_TIMEOUT;
38         do
39         {
40             /* 获取 SR1 寄存器状态 */
41             tmpSR1 = EEPROM_I2C->SR1;
42
43             if ((I2CTimeout--) == 0) return I2C_TIMEOUT_UserCallback(22);
44         }
45         /* 一直等待直到 addr 及 af 标志为 1 */
46         while ((tmpSR1 & (I2C_SR1_ADDR | I2C_SR1_AF)) == 0);
47
48         /*检查 addr 标志是否为 1 */
49         if (tmpSR1 & I2C_SR1_ADDR)
50         {
51             /* 清除 addr 标志该标志通过读 SR1 及 SR2 清除 */
52             (void)EEPROM_I2C->SR2;
53
54             /*产生停止信号 */
55             I2C_GenerateSTOP(EEPROM_I2C, ENABLE);
56
57             /* 退出函数 */
58             return 1;
59         }
60         else
61         {
62             /*清除 af 标志 */
63             I2C_ClearFlag(EEPROM_I2C, I2C_FLAG_AF);
64         }
65
66         /*检查等待次数*/
67         if (EETrials++ == MAX_TRIAL_NUMBER)
68         {
69             /* 等待 MAX_TRIAL_NUMBER 次都还没准备好, 退出等待 */
70             return I2C_TIMEOUT_UserCallback(23);
71         }
72     }
73 }
```

这个函数主要实现是向 EEPROM 发送它设备地址, 检测 EEPROM 的响应, 若 EEPROM 接收到地址后返回应答信号, 则表示 EEPROM 已经准备好, 可以开始下一次通讯。函数中检测响应是通过读取 STM32 的 SR1 寄存器的 ADDR 位及 AF 位来实现的, 当 I2C 设备响应了地址的时候, ADDR 会置 1, 若应答失败, AF 位会置 1。

EEPROM 的页写入

在以上的数据通讯中，每写入一个数据都需要向 EEPROM 发送写入的地址，我们希望向连续地址写入多个数据的时候，只要告诉 EEPROM 第一个内存地址 address1，后面的数据按次序写入到 address2、address3... 这样可以节省通讯的内容，加快速度。为应对这种需求，EEPROM 定义了一种页写入时序，见图 24-15。

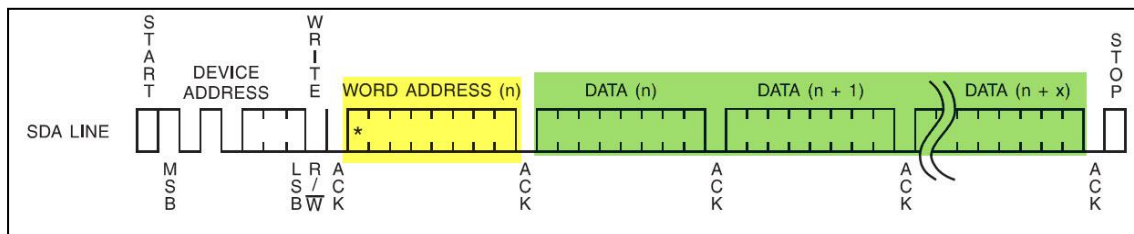


图 24-15 EEPROM 页写入时序(摘自《AT24C02》规格书)

根据页写入时序，第一个数据被解释为要写入的内存地址 address1，后续可连续发送 n 个数据，这些数据会依次写入到内存中。其中 AT24C02 型号的芯片页写入时序最多可以一次发送 8 个数据(即 n = 8)，该值也称为页大小，某些型号的芯片每个页写入时序最多可传输 16 个数据。EEPROM 的页写入代码实现见代码清单 24-8。

代码清单 24-8 EEPROM 的页写入

```
1
2 /**
3  * @brief 在 EEPROM 的一个写循环中可以写多个字节，但一次写入的字节数
4  * 不能超过 EEPROM 页的大小，AT24C02 每页有 8 个字节
5  * @param
6  * @param pBuffer:缓冲区指针
7  * @param WriteAddr:写地址
8  * @param NumByteToWrite:要写的字节数要求 NumByteToWrite 小于页大小
9  * @retval 正常返回 1，异常返回 0
10 */
11 uint8_t I2C_EE_PageWrite(uint8_t* pBuffer, uint8_t WriteAddr,
12                          uint8_t NumByteToWrite)
13 {
14     I2CTimeout = I2CT_LONG_TIMEOUT;
15
16     while (I2C_GetFlagStatus(EEPROM_I2C, I2C_FLAG_BUSY))
17     {
18         if ((I2CTimeout--) == 0) return I2C_TIMEOUT_UserCallback(4);
19     }
20
21     /* 产生 I2C 起始信号 */
22     I2C_GenerateSTART(EEPROM_I2C, ENABLE);
23
24     I2CTimeout = I2CT_FLAG_TIMEOUT;
25
26     /* 检测 EV5 事件并清除标志 */
27     while (!I2C_CheckEvent(EEPROM_I2C, I2C_EVENT_MASTER_MODE_SELECT))
28     {
29         if ((I2CTimeout--) == 0) return I2C_TIMEOUT_UserCallback(5);
30     }
31
32     /* 发送 EEPROM 设备地址 */
33     I2C_Send7bitAddress(EEPROM_I2C, EEPROM_ADDRESS, I2C_Direction_Transmitter);
34
35     I2CTimeout = I2CT_FLAG_TIMEOUT;
```

```
36
37     /* 检测 EV6 事件并清除标志 */
38     while (!I2C_CheckEvent(EEPROM_I2C,
39                             I2C_EVENT_MASTER_TRANSMITTER_MODE_SELECTED))
40     {
41         if ((I2CTimeout--) == 0) return I2C_TIMEOUT_UserCallback(6);
42     }
43     /* 发送要写入的 EEPROM 内部地址 (即 EEPROM 内部存储器的地址) */
44     I2C_SendData(EEPROM_I2C, WriteAddr);
45
46     I2CTimeout = I2CT_FLAG_TIMEOUT;
47
48     /* 检测 EV8 事件并清除标志 */
49     while (!I2C_CheckEvent(EEPROM_I2C, I2C_EVENT_MASTER_BYTE_TRANSMITTED))
50     {
51         if ((I2CTimeout--) == 0) return I2C_TIMEOUT_UserCallback(7);
52     }
53     /* 循环发送 NumByteToWrite 个数据 */
54     while (NumByteToWrite--)
55     {
56         /* 发送缓冲区中的数据 */
57         I2C_SendData(EEPROM_I2C, *pBuffer);
58
59         /* 指向缓冲区中的下一个数据 */
60         pBuffer++;
61
62         I2CTimeout = I2CT_FLAG_TIMEOUT;
63
64         /* 检测 EV8 事件并清除标志 */
65         while (!I2C_CheckEvent(EEPROM_I2C, I2C_EVENT_MASTER_BYTE_TRANSMITTED))
66         {
67             if ((I2CTimeout--) == 0) return I2C_TIMEOUT_UserCallback(8);
68         }
69     }
70     /* 发送停止信号 */
71     I2C_GenerateSTOP(EEPROM_I2C, ENABLE);
72     return 1;
73 }
```

这段页写入函数主体跟单字节写入函数是一样的，只是它在发送数据的时候，使用 for 循环控制发送多个数据，发送完多个数据后才产生 I2C 停止信号，只要每次传输的数据小于等于 EEPROM 时序规定的页大小，就能正常传输。

快速写入多字节

利用 EEPROM 的页写入方式，可以改进前面的“[多字节写入](#)”函数，加快传输速度，见代码清单 24-9。

代码清单 24-9 快速写入多字节函数

```
1
2 /* AT24C01/02 每页有 8 个字节 */
3 #define I2C_PageSize      8
4
5 /**
6  * @brief   将缓冲区中的数据写到 I2C EEPROM 中，采用页写入的方式，加快写入速度
7  * @param  pBuffer:缓冲区指针
8  * @param  WriteAddr:写地址
9  * @param  NumByteToWrite:写的字节数
10  * @retval 无
11  */
12 void I2C_EE_BufferWrite(uint8_t* pBuffer, uint8_t WriteAddr,
13                          uint16 NumByteToWrite)
14 {
```

```
15     uint8_t NumOfPage = 0, NumOfSingle = 0, Addr = 0, count = 0;
16
17     /*mod 运算求余, 若 writeAddr 是 I2C_PageSize 整数倍, 运算结果 Addr 值为 0*/
18     Addr = WriteAddr % I2C_PageSize;
19
20     /*差 count 个数据, 刚好可以对齐到页地址*/
21     count = I2C_PageSize - Addr;
22     /*计算出要写多少整数页*/
23     NumOfPage = NumByteToWrite / I2C_PageSize;
24     /*mod 运算求余, 计算出剩余不满一页的字节数*/
25     NumOfSingle = NumByteToWrite % I2C_PageSize;
26
27     /* Addr=0,则 WriteAddr 刚好按页对齐 aligned */
28     if (Addr == 0)
29     {
30         /* 如果 NumByteToWrite < I2C_PageSize */
31         if (NumOfPage == 0)
32         {
33             I2C_EE_PageWrite(pBuffer, WriteAddr, NumOfSingle);
34             I2C_EE_WaitEepromStandbyState();
35         }
36         /* 如果 NumByteToWrite > I2C_PageSize */
37         else
38         {
39             /*先把整数页都写了*/
40             while (NumOfPage--)
41             {
42                 I2C_EE_PageWrite(pBuffer, WriteAddr, I2C_PageSize);
43                 I2C_EE_WaitEepromStandbyState();
44                 WriteAddr += I2C_PageSize;
45                 pBuffer += I2C_PageSize;
46             }
47
48             /*若有多余的不满一页的数据, 把它写完*/
49             if (NumOfSingle != 0)
50             {
51                 I2C_EE_PageWrite(pBuffer, WriteAddr, NumOfSingle);
52                 I2C_EE_WaitEepromStandbyState();
53             }
54         }
55     }
56     /* 如果 WriteAddr 不是按 I2C_PageSize 对齐 */
57     else
58     {
59         /* 如果 NumByteToWrite < I2C_PageSize */
60         if (NumOfPage == 0)
61         {
62             I2C_EE_PageWrite(pBuffer, WriteAddr, NumOfSingle);
63             I2C_EE_WaitEepromStandbyState();
64         }
65         /* 如果 NumByteToWrite > I2C_PageSize */
66         else
67         {
68             /*地址不对齐多出的 count 分开处理, 不加入这个运算*/
69             NumByteToWrite -= count;
70             NumOfPage = NumByteToWrite / I2C_PageSize;
71             NumOfSingle = NumByteToWrite % I2C_PageSize;
72
73             /*先把 WriteAddr 所在页的剩余字节写了*/
74             if (count != 0)
75             {
76                 I2C_EE_PageWrite(pBuffer, WriteAddr, count);
77                 I2C_EE_WaitEepromStandbyState();
78
79                 /*WriteAddr 加上 count 后, 地址就对齐到页了*/
80                 WriteAddr += count;
```

```
81         pBuffer += count;
82     }
83     /*把整数页都写了*/
84     while (NumOfPage--)
85     {
86         I2C_EE_PageWrite(pBuffer, WriteAddr, I2C_PageSize);
87         I2C_EE_WaitEepromStandbyState();
88         WriteAddr += I2C_PageSize;
89         pBuffer += I2C_PageSize;
90     }
91     /*若有多余的不满一页的数据，把它写完*/
92     if (NumOfSingle != 0)
93     {
94         I2C_EE_PageWrite(pBuffer, WriteAddr, NumOfSingle);
95         I2C_EE_WaitEepromStandbyState();
96     }
97 }
98 }
99 }
```

很多读者觉得这段代码的运算很复杂，看不懂，其实它的主旨就是对输入的数据进行分页(本型号芯片每页 8 个字节)，见表 24-2。通过“整除”计算要写入的数据 NumByteToWrite 能写满多少“完整的页”，计算得的值存储在 NumOfPage 中，但有时数据不是刚好能写满完整页的，会多一点出来，通过“求余”计算得出“不满一页的数据个数”就存储在 NumOfSingle 中。计算后通过按页传输 NumOfPage 次整页数据及最后的 NumOfSing 个数据，使用页传输，比之前的单个字节数据传输要快很多。

除了基本的分页传输，还要考虑首地址的问题，见表 24-3。若首地址不是刚好对齐到页的首地址，会需要一个 count 值，用于存储从该首地址开始写满该地址所在的页，还能写多少个数据。实际传输时，先把这部分 count 个数据先写入，填满该页，然后把剩余的数据(NumByteToWrite-count)，再重复上述求出 NumOPage 及 NumOfSingle 的过程，按页传输到 EEPROM。

1. 若 writeAddress=16，计算得 Addr=16%8= 0，count=8-0= 8；
2. 同时，若 NumOfPage=22，计算得 NumOfPage=22/8= 2，NumOfSingle=22%8= 6。

数据传输情况如表 24-2。

表 24-2 首地址对齐到页时的情况

不影响	0	1	2	3	4	5	6	7
不影响	8	9	10	11	12	13	14	15
第 1 页	16	17	18	19	20	21	22	23
第 2 页	24	25	26	27	28	29	30	31
NumOfSingle=6	32	33	34	35	36	37	38	39

3. 若 writeAddress=17，计算得 Addr=17%8= 1，count=8-1= 7；
4. 同时，若 NumOfPage=22，
5. 先把 count 去掉，特殊处理，计算得新的 NumOfPage=22-7= 15
6. 计算得 NumOfPage=15/8= 1，NumOfSingle=15%8= 7。

数据传输情况如表 24-3。

表 24-3 首地址未对齐到页时的情况

不影响	0	1	2	3	4	5	6	7
不影响	8	9	10	11	12	13	14	15
count=7	16	17	18	19	20	21	22	23
第 1 页	24	25	26	27	28	29	30	31
NumOfSingle=7	32	33	34	35	36	37	38	39

最后，强调一下，EEPROM 支持的页写入只是一种加速的 I2C 的传输时序，实际上并不要求每次都以页为单位进行读写，EEPROM 是支持随机访问的(直接读写任意一个地址)，如前面的单个字节写入。在某些存储器，如 NAND FLASH，它是必须按照 Block 写入的，例如每个 Block 为 512 或 4096 字节，数据写入的最小单位是 Block，写入前都需要擦除整个 Block；NOR FLASH 则是写入前必须以 Sector/Block 为单位擦除，然后才可以按字节写入。而我们的 EEPROM 数据写入和擦除的最小单位是“字节”而不是“页”，数据写入前不需要擦除整页。

从 EEPROM 读取数据

从 EEPROM 读取数据是一个复合的 I2C 时序，它实际上包含一个写过程和一个读过程，见图 24-16。

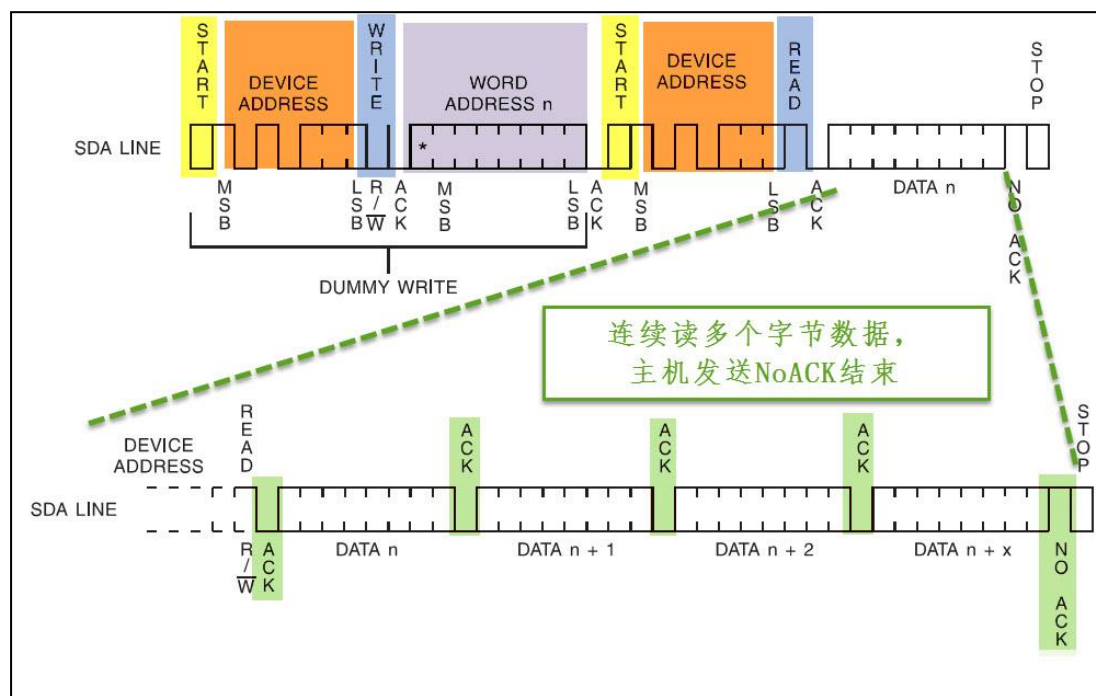


图 24-16 EEPROM 数据读取时序

读时序的第一个通讯过程中，使用 I2C 发送设备地址寻址(写方向)，接着发送要读取的“内存地址”；第二个通讯过程中，再次使用 I2C 发送设备地址寻址，但这个时候的数据方向是读方向；在这个过程之后，EEPROM 会向主机返回从“内存地址”开始的数据，一个字节一个字节地传输，只要主机的响应为“应答信号”，它就会一直传输下去，主机想结束传输时，就发送“非应答信号”，并以“停止信号”结束通讯，作为从机的 EEPROM 也会停止传输。实现代码见代码清单 24-10。

代码清单 24-10 从 EEPROM 读取数据

```
1
2 /**
3  * @brief 从 EEPROM 里面读取一块数据
4  * @param pBuffer:存放从 EEPROM 读取的数据的缓冲区指针
5  * @param ReadAddr:接收数据的 EEPROM 的地址
6  * @param NumByteToRead:要从 EEPROM 读取的字节数
7  * @retval 正常返回 1, 异常返回 0
8  */
9 uint8_t I2C_EE_BufferRead(uint8_t* pBuffer, uint8_t ReadAddr,
10                           u16 NumByteToRead)
11 {
12     I2CTimeout = I2CT_LONG_TIMEOUT;
13
14     while (I2C_GetFlagStatus(EEPROM_I2C, I2C_FLAG_BUSY))
15     {
16         if ((I2CTimeout--) == 0) return I2C_TIMEOUT_UserCallback(9);
17     }
18
19     /* 产生 I2C 起始信号 */
20     I2C_GenerateSTART(EEPROM_I2C, ENABLE);
21
22     I2CTimeout = I2CT_FLAG_TIMEOUT;
23
24     /* 检测 EV5 事件并清除标志 */
25     while (!I2C_CheckEvent(EEPROM_I2C, I2C_EVENT_MASTER_MODE_SELECT))
26     {
27         if ((I2CTimeout--) == 0) return I2C_TIMEOUT_UserCallback(10);
28     }
29
30     /* 发送 EEPROM 设备地址 */
31     I2C_Send7bitAddress(EEPROM_I2C, EEPROM_ADDRESS, I2C_Direction_Transmitter);
32
33     I2CTimeout = I2CT_FLAG_TIMEOUT;
34
35     /* 检测 EV6 事件并清除标志 */
36     while (!I2C_CheckEvent(EEPROM_I2C,
37                             I2C_EVENT_MASTER_TRANSMITTER_MODE_SELECTED))
38     {
39         if ((I2CTimeout--) == 0) return I2C_TIMEOUT_UserCallback(11);
40     }
41     /*通过重新设置 PE 位清除 EV6 事件 */
42     I2C_Cmd(EEPROM_I2C, ENABLE);
43
44     /* 发送要读取的 EEPROM 内部地址(即 EEPROM 内部存储器的地址) */
45     I2C_SendData(EEPROM_I2C, ReadAddr);
46
47     I2CTimeout = I2CT_FLAG_TIMEOUT;
48
49     /* 检测 EV8 事件并清除标志 */
50     while (!I2C_CheckEvent(EEPROM_I2C, I2C_EVENT_MASTER_BYTE_TRANSMITTED))
51     {
52         if ((I2CTimeout--) == 0) return I2C_TIMEOUT_UserCallback(12);
53     }
54     /* 产生第二次 I2C 起始信号 */
55     I2C_GenerateSTART(EEPROM_I2C, ENABLE);
56
57     I2CTimeout = I2CT_FLAG_TIMEOUT;
58
59     /* 检测 EV5 事件并清除标志 */
60     while (!I2C_CheckEvent(EEPROM_I2C, I2C_EVENT_MASTER_MODE_SELECT))
61     {
62         if ((I2CTimeout--) == 0) return I2C_TIMEOUT_UserCallback(13);
63     }
64     /* 发送 EEPROM 设备地址 */
65     I2C_Send7bitAddress(EEPROM_I2C, EEPROM_ADDRESS, I2C_Direction_Receiver);
66 }
```

```
67     I2CTimeout = I2CT_FLAG_TIMEOUT;
68
69     /* 检测 EV6 事件并清除标志 */
70     while (!I2C_CheckEvent(EEPROM_I2C,
71                             I2C_EVENT_MASTER_RECEIVER_MODE_SELECTED))
72     {
73         if ((I2CTimeout-- == 0) return I2C_TIMEOUT_UserCallback(14);
74     }
75     /* 读取 NumByteToRead 个数据 */
76     while (NumByteToRead)
77     {
78         /* 若 NumByteToRead=1, 表示已经接收到最后一个数据了,
79         发送非应答信号, 结束传输 */
80         if (NumByteToRead == 1)
81         {
82             /* 发送非应答信号 */
83             I2C_AcknowledgeConfig(EEPROM_I2C, DISABLE);
84
85             /* 发送停止信号 */
86             I2C_GenerateSTOP(EEPROM_I2C, ENABLE);
87         }
88
89         I2CTimeout = I2CT_LONG_TIMEOUT;
90         while (I2C_CheckEvent(EEPROM_I2C, I2C_EVENT_MASTER_BYTE_RECEIVED) == 0)
91         {
92             if ((I2CTimeout-- == 0) return I2C_TIMEOUT_UserCallback(3);
93         }
94         {
95             /* 通过 I2C, 从设备中读取一个字节的的数据 */
96             *pBuffer = I2C_ReceiveData(EEPROM_I2C);
97
98             /* 存储数据的指针指向下一个地址 */
99             pBuffer++;
100
101             /* 接收数据自减 */
102             NumByteToRead--;
103         }
104     }
105
106     /* 使能应答, 方便下一次 I2C 传输 */
107     I2C_AcknowledgeConfig(EEPROM_I2C, ENABLE);
108     return 1;
109 }
```

这段中的写过程跟前面的写字节函数类似, 而读过程中接收数据时, 需要使用库函数 I2C_ReceiveData 来读取。响应信号则通过库函数 I2C_AcknowledgeConfig 来发送, DISABLE 时为非响应信号, ENABLE 为响应信号。

3. main 文件

EEPROM 读写测试函数

完成基本的读写函数后, 接下来我们编写一个读写测试函数来检验驱动程序, 见代码清单 24-11。

代码清单 24-11 EEPROM 读写测试函数

```
1 /**
2  * @brief I2C(AT24C02) 读写测试
3  * @param 无
4  * @retval 正常返回 1, 不正常返回 0
5  */
```

```
6 uint8_t I2C_Test(void)
7 {
8     u16 i;
9     EEPROM_INFO("写入的数据");
10
11     for ( i=0; i<=255; i++ ) //填充缓冲
12     {
13         I2c_Buf_Write[i] = i;
14
15         printf("0x%02X ", I2c_Buf_Write[i]);
16         if (i%16 == 15)
17             printf("\n\r");
18     }
19
20     //将 I2c_Buf_Write 中顺序递增的数据写入 EEPROM 中
21     //页写入方式
22     // I2C_EE_BufferWrite( I2c_Buf_Write, EEP_Firstpage, 256);
23     //字节写入方式
24     I2C_EE_ByetsWrite( I2c_Buf_Write, EEP_Firstpage, 256);
25
26     EEPROM_INFO("写结束");
27
28     EEPROM_INFO("读出的数据");
29     //将 EEPROM 读出数据顺序保持到 I2c_Buf_Read 中
30     I2C_EE_BufferRead(I2c_Buf_Read, EEP_Firstpage, 256);
31
32     //将 I2c_Buf_Read 中的数据通过串口打印
33     for (i=0; i<256; i++)
34     {
35         if (I2c_Buf_Read[i] != I2c_Buf_Write[i])
36         {
37             printf("0x%02X ", I2c_Buf_Read[i]);
38             EEPROM_ERROR("错误:I2C EEPROM 写入与读出的数据不一致");
39             return 0;
40         }
41         printf("0x%02X ", I2c_Buf_Read[i]);
42         if (i%16 == 15)
43             printf("\n\r");
44     }
45
46     EEPROM_INFO("I2C(AT24C02) 读写测试成功");
47     return 1;
48 }
```

代码中先填充一个数组，数组的内容为 1,2,3 至 N，接着把这个数组的内容写入到 EEPROM 中，写入时可以采用单字节写入的方式或页写入的方式。写入完毕后再从 EEPROM 的地址中读取数据，把读取得到的与写入的数据进行校验，若一致说明读写正常，否则读写过程有问题或者 EEPROM 芯片不正常。其中代码用到的 EEPROM_INFO 跟 EEPROM_ERROR 宏类似，都是对 printf 函数的封装，使用和阅读代码时把它直接当成 printf 函数就好。具体的宏定义在“bsp_i2c_ee.h 文件中”，在以后的代码我们常常会用类似的宏来输出调试信息。

main 函数

最后编写 main 函数，函数中初始化了 LED、串口、I2C 外设，然后调用上面的 I2C_Test 函数进行读写测试，见代码清单 24-12。

代码清单 24-12 main 函数

```
1
2 /**
```

```
3  * @brief 主函数
4  * @param 无
5  * @retval 无
6  */
7  int main(void)
8  {
9      LED_GPIO_Config();
10
11     LED_BLUE;
12     /*初始化 USART1*/
13     Debug_USART_Config();
14
15     printf("\r\n 欢迎使用野火  STM32 F407 开发板.\r\n");
16
17     printf("\r\n 这是一个 I2C 外设(AT24C02)读写测试例程 \r\n");
18
19     /* I2C 外设(AT24C02)初始化 */
20     I2C_EE_Init();
21
22     if (I2C_Test() ==1)
23     {
24         LED_GREEN;
25     }
26     else
27     {
28         LED_RED;
29     }
30
31     while (1)
32     {
33     }
34 }
35 }
36
```

24.4.3 下载验证

用 USB 线连接开发板“USB TO UART”接口跟电脑，在电脑端打开串口调试助手，把编译好的程序下载到开发板。在串口调试助手可看到 EEPROM 测试的调试信息。